AI-Powered Project Evaluation Platform — Backend Architecture

Assignment: SDE-II Backend Architecture, AlmaBetter **Author:** Tarun Meena **Date:** 12 October 2026
Companion repository: `project-evaluator/` (proof of concept: API, worker, LLM pipeline, schema, tests). File references below such as `src/pipeline/stages.ts` point into it.

---

0. How to read this document

Each numbered section answers one section of the brief in order. Two threads run through all of them:

1. **What I would build today** for 30 submissions/day. It is small on purpose: a modular monolith (one codebase, two processes), PostgreSQL, object storage, one managed LLM provider behind an interface.
2. **What changes at 10,000/day**, and more importantly *why* and *when*. Most changes are triggered by a measurable signal (queue age, cost per evaluation, p95 latency), not by a date.

The proof of concept exists to keep me honest: every mechanism the document relies on (checkpointed stages, lease-based crash recovery, idempotent start, rubric versioning, state-machine guards, safe extraction, structured LLM outputs, review routing) is implemented and exercised by `npm run demo` and `npm test`. Where the PoC simplifies something (local disk instead of S3, Postgres queue instead of SQS, lexical retrieval instead of embeddings), the document says so.

Executive summary

- **Shape:** API (Fastify/TypeScript) + stateless workers pulling stage jobs from a queue + PostgreSQL as the system of record + S3 for files + Claude through a provider interface. The LLM never sees a raw project; it sees a *project card* built map-reduce style from bounded file summaries, and it scores **one rubric criterion per call** with schema-enforced JSON and cited evidence.
- **Reliability model:** every evaluation is a state machine persisted in Postgres; every stage is an idempotent job with a lease; every unsafe POST is idempotent by key *and* by a database uniqueness invariant. Crashes, duplicate deliveries and LLM failures are the normal path, not exceptions.
- **The real scaling constraint is cost, not compute.** At 10k/day the LLM bill (roughly $0.60–1.00 per evaluation) dwarfs infrastructure. The architecture therefore invests in token economy (model tiering, prompt caching, content-hash reuse, batch API) more than in Kubernetes.
- **Security posture:** student uploads are hostile by default. Nothing is ever executed in Phase 1; extraction is streamed and bounded; URLs are SSRF-filtered; the LLM is told the material is untrusted and the pipeline detects and flags injection attempts; low-confidence or flagged results go to a human.

---

1. Requirements and capacity model

1.1 Functional scope

Actor	Needs
Student	Create a submission from one or more sources (GitHub, ZIP, PDF, Drive, video, docs, live URL); start evaluation; see status, score, per-criterion feedback; request a human review.
Evaluator	Queue of evaluations needing review; see AI rationale and evidence; approve/reject; override scores.
Course team / admin	Define and version rubrics per assignment; see throughput, failures, costs.
System	Process evaluations asynchronously (2–15 min), survive worker and LLM failures, never double-process, keep a reproducible history.

1.2 Non-functional targets (Phase 1)

Attribute	Target
Time to result	p50 < 8 min, p95 < 20 min after upload completes
Durability	No lost submissions or results (RDS Multi-AZ, S3)
Correctness	No evaluation ever runs twice concurrently for one submission; results always match a pinned rubric version
Availability	API 99.9% monthly; evaluation pipeline may degrade (queue grows) without data loss
Cost	< $1.00 LLM cost per evaluation before optimisation; visible per evaluation

1.3 Capacity arithmetic

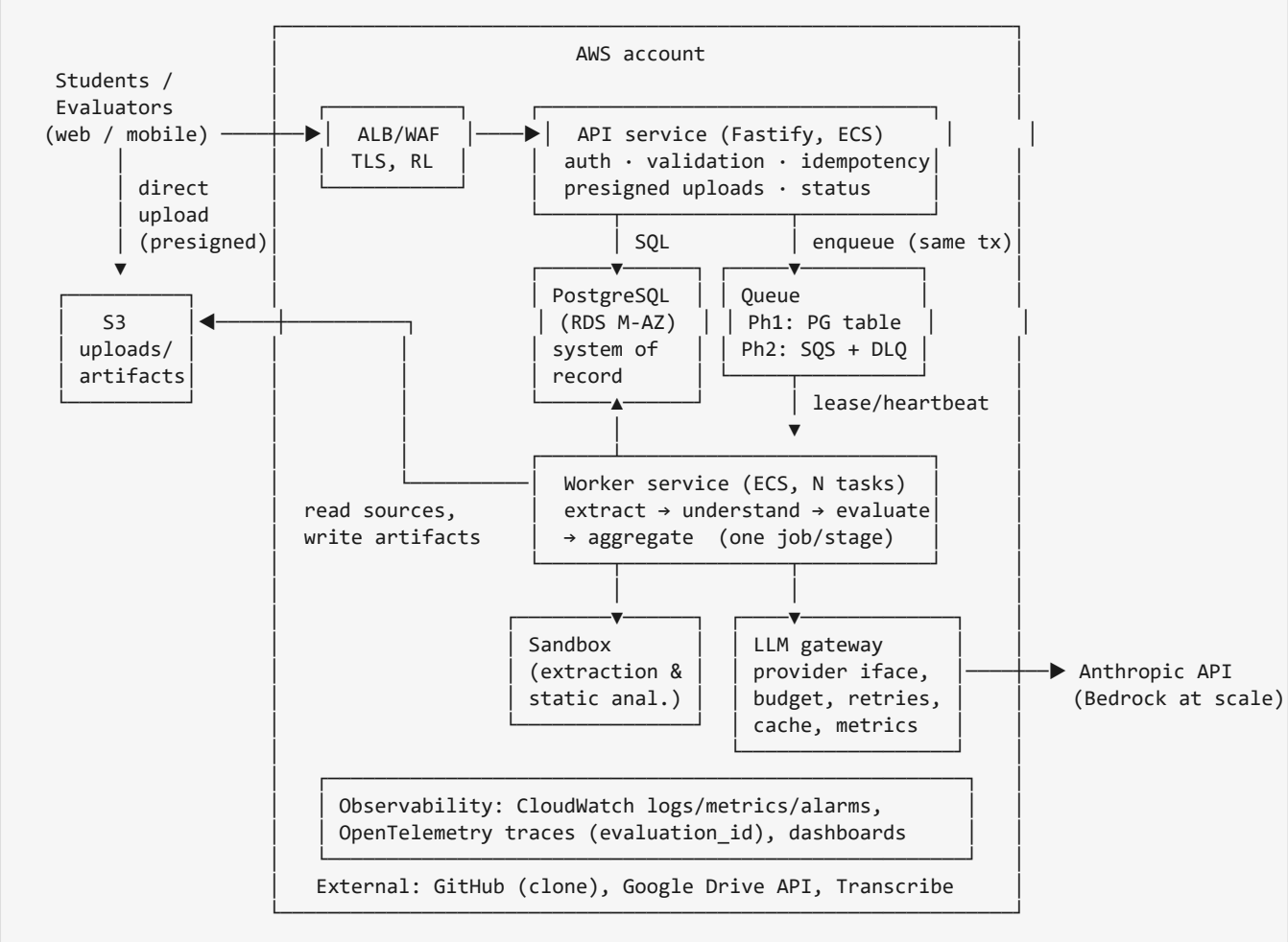
Quantity	30/day (Phase 1)	10,000/day (Phase 3)
Average arrival rate	1.25/hour	~7/minute
Deadline peak (assume 5× average over a few hours)	6/hour	~35/minute
Evaluation duration (avg / max)	8 min / 15 min	same (LLM-bound)
Concurrent evaluations needed (rate × duration)	< 1 avg, ~1 peak	~56 avg, ~280 peak
Worker instances (≈8 concurrent LLM-bound evaluations each)	1–2 (for redundancy)	7 avg, 35 peak, autoscaled
Raw upload storage (avg 100 MB)	3 GB/day, ~1 TB/year	1 TB/day → lifecycle to cold storage mandatory
LLM tokens per evaluation (see §6.6)	~420k input / ~28k output	same per evaluation
LLM calls per evaluation	~50	same; 500k calls/day → provider rate limits become a design input

The two numbers that change the architecture are the **concurrency (~280 at peak)**, which forces separate scaling of extraction (disk/CPU) and LLM (I/O) work, and the **LLM volume (≈4 billion input tokens/day)**, which forces the cost work in §9.

---

2. System architecture

2.1 High-level diagram (Phase 1)



2.2 Components and why

Layer	Choice (Phase 1)	Why this and not the alternatives
Client	Web SPA (React) + student dashboard; evaluator console	Out of scope for the backend design; the API is the contract. Files upload **directly to S3** via presigned URLs so the API never proxies gigabytes.
API	**Node.js 22 + TypeScript + Fastify**	Fastify is fast, schema-friendly and has first-class plugins for JWT and multipart. TypeScript across API and workers means one language, shared types for jobs/schemas, and Zod used both for request validation and for LLM structured-output schemas. Python was the alternative (richer parsing/ML libraries); I chose not to go polyglot at this size and to keep the ML-flavoured work in the LLM provider rather than in local libraries.
Auth/authz	JWT (12h) issued by the API after password login; roles `student / evaluator / admin`; row-level ownership checks in every query	Simple, stateless, and sufficient for three roles. Phase 2: an IdP (Cognito/Auth0/Keycloak) with the same role claims, refresh tokens, and SSO for staff.

Layer	Choice (Phase 1)	Why this and not the alternatives
Backend services	**Modular monolith**: one codebase, two deployables (`api`, `worker`)	30/day does not justify microservices; separate *processes* already give independent scaling and failure isolation. Modules (`api/`, `pipeline/`, `llm/`, `queue/`, `storage/`, `domain/`) have explicit interfaces so a module can become a service later without a rewrite.
Database	**PostgreSQL 16** (RDS Multi-AZ)	Relational data with strong invariants (one active evaluation per submission, pinned rubric versions, state transitions). Partial unique indexes, transactions and `SKIP LOCKED` do real work here. JSONB covers the semi-structured bits (evidence, flags, project card). See §11.
Object storage	**S3** (local disk in the PoC behind `ObjectStore`)	Uploads, extracted artifacts, evaluation checkpoints and reports. Versioning off, SSE-S3/KMS on, lifecycle rules to cold storage.
Queue	**Postgres job table** (Phase 1) → **SQS standard + DLQ** (Phase 2) behind `JobQueue`	At 30/day a Postgres queue (`FOR UPDATE SKIP LOCKED`) gives transactional enqueue (job + state change commit together) with zero extra infrastructure. SQS is the planned step when volume or isolation demands it; the interface already mirrors SQS semantics (visibility timeout = lease, max receive count = dead letter).
Background workers	Stateless ECS tasks running the same image with a different entrypoint	Each stage is one job; workers keep no state between jobs (artifacts go to S3) so any worker can resume any evaluation.
AI/LLM layer	**Claude via the Anthropic SDK** behind `LlmProvider`; `claude-opus-5-5` for judgement, `claude-haiku-5-5` for bulk summaries; a deterministic `MockProvider` for tests/CI	Structured outputs (schema-enforced JSON), prompt caching, 1M context and a batch API map directly onto the pipeline's needs. The interface isolates vendor specifics so Bedrock (same models, IAM auth, in-VPC) or another vendor is a config change.
Evaluation engine	`pipeline/`: inventory → summaries (map) → project card (reduce) → per-criterion scoring → aggregation + review policy	Rubric criteria are **data** (`rubric_criteria` rows) rendered into the prompt; the engine knows nothing about specific criteria.
Monitoring/logging	pino JSON logs → CloudWatch; metrics on queue age, stage durations, failures by code, LLM tokens/cost; OpenTelemetry traces keyed by `evaluation_id`	Debugging an async pipeline is impossible without a correlation id that spans API → queue → worker → LLM call.
External integrations	GitHub (anonymous shallow clone; GitHub App for private repos), Google Drive API, Amazon Transcribe/Whisper for video, headless probe for live URLs	Each is an adapter producing files or metadata into the same extraction root, so the rest of the pipeline is source-agnostic.

2.3 What is deliberately *not* in Phase 1

- No Redis: rate limiting lives at the ALB/WAF and the API; status is polled. Redis arrives with SSE fan-out and a cross-worker LLM rate limiter (§9).
- No vector database: evidence retrieval is lexical over file summaries. pgvector is the first step when it is needed (§6.3).
- No code execution: Functionality is judged from tests present, README claims, video transcript and a safe probe of the live URL. Sandboxed test execution is a Phase 2 feature (§8.4).

3. Submission processing pipeline

3.1 Lifecycle

```
Student                    API (sync)                                  Worker (async)
─────                      ──────────                                  ──────────────
POST /submissions ──────▶ validate sources, create submission
                          + submission_sources (draft)
                          return presigned upload URLs
PUT file → S3 ──────────▶ (direct to S3; API not involved)
POST .../upload complete ─▶ verify size/sha/magic bytes, mark source stored
POST .../evaluations ─────▶ idempotency check
  (Idempotency-Key)       BEGIN
                            lock submission; all sources stored?
                            rubric = assignment.active_rubric (published)
                            INSERT evaluations(status=queued, rubric_id)   ← unique active/submission
                            INSERT jobs(extract) / SQS send               ← same transaction (outbox)
                            submission.status = evaluating
                          COMMIT → 202 {evaluation}
                                                                    lease job(extract)
                                                                    transition queued→extracting
                                                                    materialise sources (zip/clone)
                                                                    write extract manifest → S3
                                                                    enqueue(understand) · complete job
                                                                    lease job(understand)
                                                                    inventory → file summaries → project
                                                                    write artifacts → S3
                                                                    enqueue(evaluate) · complete job
                                                                    lease job(evaluate)
                                                                    per criterion: retrieve evidence → LL
                                                                    enqueue(aggregate) · complete job
                                                                    lease job(aggregate)
                                                                    weighted total, review policy, final
                                                                    BEGIN results + status + review row +
GET /evaluations/:id ─────▶ status/progress/attempts (poll; SSE later)
GET /evaluations/:id/result▶ scores, rationale, evidence, review
```

3.2 Synchronous vs asynchronous

Operation	Mode	Reason
Create submission, validate URLs, issue upload URLs	Sync	Milliseconds; the student needs the answer to proceed.
File upload	Sync from the browser, **direct to S3**	100 MB–1 GB must not flow through API instances (memory, timeouts, cost). Presigned multipart PUT; completion callback verifies size and checksum.
Start evaluation	Sync *acknowledgement* (202), async *work*	The API only validates and enqueues inside one transaction. Returning 202 with an id is the contract.
Extract / understand / evaluate / aggregate	Async, one queue message per stage	2–15 minutes, many LLM calls, must survive restarts.
Status, result, notifications	Sync reads	Served from Postgres; cheap.
Rubric update, review decision	Sync	Small transactions; the decision endpoint recomputes the aggregate in-transaction (`src/api/routes/evaluations.ts`).

3.3 Where queues are required, and how workers communicate

- **One queue, four stage types** (`evaluation-stages` with payload `{evaluationId, stage}`) in Phase 1. Stage completion enqueues the next stage's message. Workers never talk to each other; **they communicate only through the database (state, scores) and object storage (artifacts)**. This is what makes workers stateless and interchangeable.
- Why per-stage messages instead of one long job: (1) a crash resumes from the last completed stage rather than minute zero; (2) stages have different resource profiles (extract: disk + CPU, evaluate: network I/O), so at scale they get separate queues and worker pools; (3) leases stay short enough to detect dead workers quickly.
- Checkpoints inside stages too: `understand` persists summaries before building the project card and reuses them on retry; `evaluate` upserts one row per criterion and skips criteria already scored (`ON CONFLICT (evaluation_id, criterion_id)`), so a retry after scoring 4 of 6 criteria only pays for the remaining 2.

3.4 Tracking status

`evaluations.status` is the single source of truth, changed only through `transition()` (`src/domain/evaluation-state.ts`), a compare-and-set `UPDATE … WHERE status = ANY(expected)`. `progress` is set per stage (10/35/60/90/100). `evaluation_attempts` records every stage attempt with worker id, timing, error and token usage, which is what `GET /evaluations/:id` shows and what support reads when a student asks "what happened".

3.5 Failure walk-throughs

Worker crashes mid-stage. The job is leased with a visibility timeout (PoC: `leased_until`; SQS: visibility timeout). The worker heartbeats every third of the lease while working. If it dies, the lease expires, the reaper (or SQS) returns the message to the queue, another worker picks it up with `attempts = 2`, and the stage handler re-runs. Handlers are idempotent: extraction is deterministic, summaries are reused from S3, criterion rows upsert, the final write is one transaction. The state machine allows a stage status to re-enter itself for exactly this reason.

LLM API fails. Classified in `src/llm/anthropic.ts` into retryable (429, 5xx/529 overloaded, timeouts, network) and non-retryable (400, auth, refusal, truncated output). Retryable errors get bounded exponential backoff with jitter inside the call (`BudgetedLlm`), then the *stage* fails and is retried by the queue with its own backoff, then the evaluation is marked `failed` with an error code after `max_attempts`. Non-retryable errors fail fast; a *refusal* on one criterion does not fail the evaluation: it records a zero-confidence placeholder and forces human review. Every evaluation has a hard cap on LLM calls (`LLM_MAX_CALLS_PER_EVALUATION`) so a bug cannot burn budget in a loop.

Same submission processed twice. Four independent guards, from outermost to innermost: (1) `Idempotency-Key` replay (§7.3); (2) the partial unique index `evaluations_one_active_per_submission` makes a second active evaluation impossible at the database level, so even two API instances racing produce one row and one 409; (3) job deduplication keys (`<evaluationId>:<stage>`) make enqueueing the same stage twice a no-op; (4) state-machine guards make a late or duplicate delivery a no-op (the worker logs it and acknowledges the message). Results are written with `ON CONFLICT … DO UPDATE`, so even if everything above failed the data would still be consistent.

---

4. API design

Conventions: JSON; `Authorization: Bearer <jwt>`; errors as `{ "error": { "code", "message", "details?" } }`; ids are UUIDs; all writes are validated with Zod schemas (`src/api/routes/*.ts`). Students can only see their own submissions and evaluations; evaluators and admins see all. Pagination is `limit` + cursor (PoC: `limit` only).

#	Method & path	Who	Purpose
1	`POST /auth/login`	anyone	Exchange credentials for a JWT
2	`GET /assignments`	any role	List assignments with active rubric id
3	`GET /assignments/:id/rubric`	any role	Active published rubric with criteria
4	`PUT /assignments/:id/rubric`	evaluator, admin	Publish a **new rubric version**
5	`POST /submissions`	student	Create submission; returns upload instructions
6	`POST /submissions/:id/sources/:sid/upload`	owner	Upload ZIP/PDF/docs (PoC: multipart to API; prod: presigned S3 + completion callback)
7	`GET /submissions/:id` · `GET /submissions`	owner / staff	Submission with sources and evaluation history
8	`POST /submissions/:id/evaluations`	owner, staff	**Start evaluation** (idempotent) → 202
9	`GET /evaluations/:id`	owner, staff	Status, progress, attempts, errors
10	`GET /evaluations/:id/result`	owner, staff	Scores, rationale, evidence, review (202 while running)
11	`POST /evaluations/:id/review-requests`	owner, staff	Request human review
12	`POST /evaluations/:id/decision`	evaluator, admin	Approve / reject, optional score overrides
13	`GET /reviews`	evaluator, admin	Open review queue
14	`GET /notifications` · `POST /notifications/:id/read`	any role	Student dashboard feed
15	`GET /healthz` · `GET /readyz` · `GET /admin/queue`	public / admin	Liveness, readiness, queue and DLQ visibility

4.1 Create submission

```
POST /submissions
Authorization: Bearer <student jwt>
{
  "assignment_id": "c7e5c4ef-…",
  "title": "Task Manager API",
  "notes": "Capstone submission",
  "sources": [
    { "kind": "github", "url": "https://github.com/asha/task-manager" },
    { "kind": "zip", "filename": "project.zip" },
    { "kind": "video", "url": "https://youtu.be/…" },
    { "kind": "live_url", "url": "https://task-manager.fly.dev" }
  ]
}
→ 201
{
  "submission": { "id": "…", "status": "draft", "sources": [ {"id":"…","kind":"github","status":"stored"},
  "uploads": [ { "source_id": "…", "method": "PUT", "url": "https://s3…/presigned", "expires_in": 900, "max
}
```

Validation: 1–10 sources; `github` must match `https://github.com/{owner}/{repo}` exactly (no SSH, no credentials, no sub-paths); `live_url/video/drive` must be `http(s)` without credentials and must not resolve to loopback, RFC1918, link-local (169.254/16 metadata), or `.internal/.local` hosts (`src/api/url-safety.ts`); upload kinds must not carry a URL; title 1–200 chars; the assignment must exist and have a published rubric before evaluation (checked at start, not at create, so students can save drafts).

4.2 Upload completion (production shape)

```
POST /submissions/:id/sources/:sid/complete      { "etag": "…", "size_bytes": 104857600 }
→ 200 { "source": { "status": "stored", "sha256": "…" } }
```

The API HEADs the object, checks size ≤ limit and declared kind vs magic bytes (ZIP `50 4B 03 04`, PDF `%PDF`), computes/validates the checksum asynchronously for large objects, and only then marks the source `stored`. The PoC accepts the multipart body directly and performs the same checks (`src/api/routes/submissions.ts`).

4.3 Start evaluation

```
POST /submissions/:id/evaluations
Authorization: Bearer <jwt>
Idempotency-Key: 7f3c…  (recommended)
→ 202 { "evaluation": { "id": "…", "status": "queued", "progress": 0, "rubric_id": "…" } }
→ 202 + Idempotent-Replayed: true     (same key, same body)
→ 409 evaluation_in_progress          (another active evaluation exists; body includes its id)
→ 409 sources_pending                 (some upload not completed)
→ 409 no_rubric                       (assignment has no published rubric)
→ 422 idempotency_key_reused          (same key, different body)
```

4.4 Status and result

```
GET /evaluations/:id
→ 200 { "id", "status": "evaluating", "progress": 60, "attempt_count": 3, "error_code": null,
        "attempts": [ { "stage": "extract", "attempt": 1, "status": "succeeded", "llm_calls": 0, … },
                      { "stage": "understand", "attempt": 1, "status": "failed", "error": "llm_rate_limit: .
                      { "stage": "understand", "attempt": 2, "status": "succeeded", "llm_calls": 41, … } ]

GET /evaluations/:id/result
→ 202 { "status": "evaluating", "progress": 60 }          while not finished
→ 200 { "result": { "percentage": 78.75, "total_score": 78.75, "max_total": 100, "confidence": 0.75,
                    "summary": "…", "strengths": [...], "improvements": [...], "flags": [] },
        "criteria": [ { "key": "functionality", "score": 8, "max_score": 10, "weight": 25, "confidence": 0.8
                        "rationale": "…", "evidence": [ { "path": "tests/tasks.test.js", "excerpt": "…", "n
        "human_review": null | { "status": "approved", "decision_comment": "…", "score_overrides": {...} }
```

Students see the result only when `status ∈ {completed, approved}`; `needs_review` results are visible to staff and shown to the student as "under review" (policy choice, enforced in the route).

4.5 Update rubric

```
PUT /assignments/:id/rubric            (evaluator | admin)
{ "criteria": [ { "key": "functionality", "name": "Functionality", "weight": 25, "max_score": 10,
                  "description": "…", "guidance": "10: … 6: … 3: … 0: …" }, … ] }
→ 201 { "id": "<new rubric id>", "version": 3, "status": "published", "criteria": [...] }
```

Validation: keys snake_case and unique; weights ≥ 0 and **sum to 100 ± 0.01**; 1–30 criteria. Behaviour: inserts a new `rubrics` row with `version = max+1`, archives the previous published version, points `assignments.active_rubric_id` at the new one. Running evaluations keep their pinned `rubric_id`.

4.6 Human review

```
POST /evaluations/:id/review-requests   { "reason": "The functionality score ignores the video demo" }
→ 201 { "review": { "status": "requested", … } }            (evaluation: completed → needs_review)

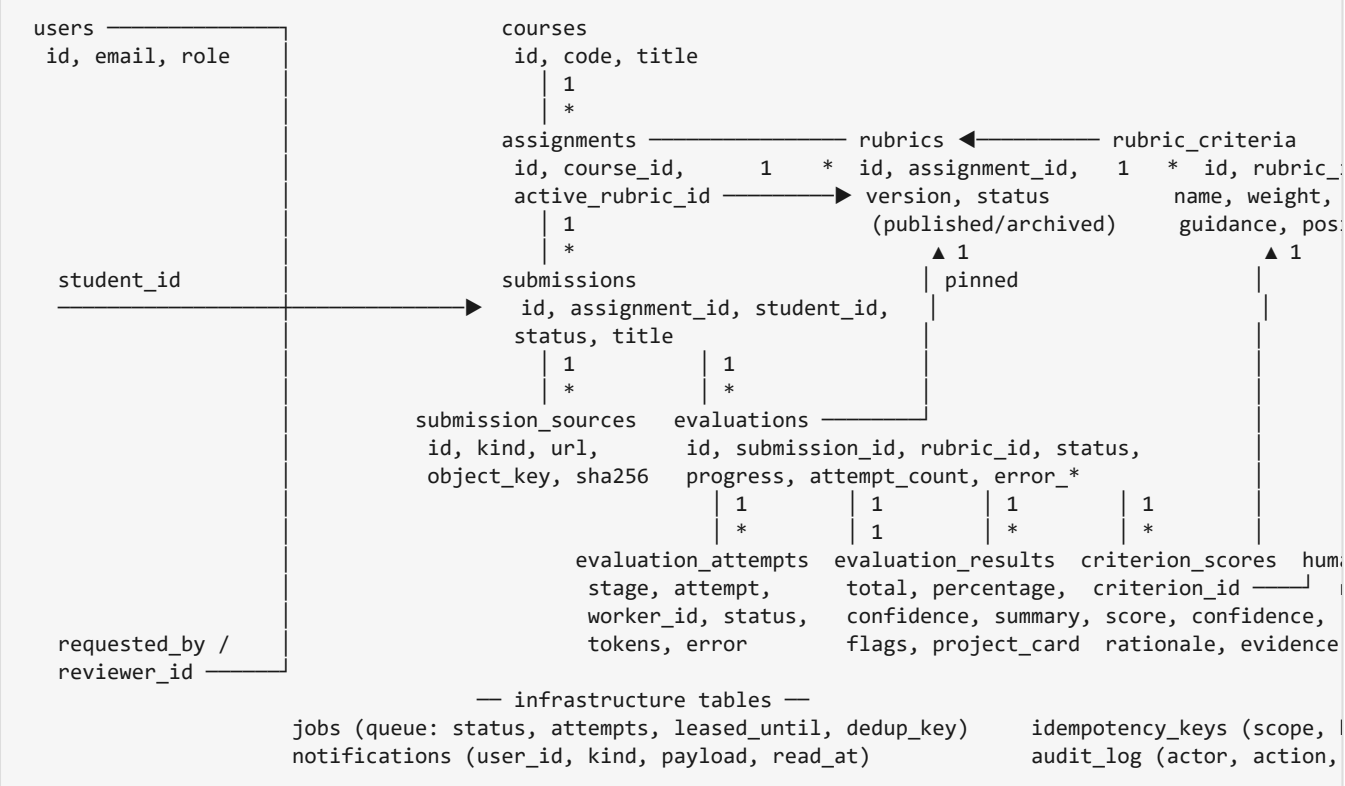
POST /evaluations/:id/decision          (evaluator | admin)
{ "decision": "approve", "comment": "Re-checked tests; raising functionality.", "score_overrides": { "funct
→ 200 { "evaluation": { "status": "approved" }, "review": {...}, "result": { "percentage": 81.25, … } }
```

Overrides are applied to `criterion_scores` (confidence → 1, flag `human_override`), the aggregate is recomputed in the same transaction, the review row is closed, the submission becomes `finalized` (approve) or returns to `submitted` (reject, eligible for re-evaluation), and the student is notified.

---

5. Database design

5.1 ER diagram



The full DDL with comments is `db/migrations/001_init.sql`.

5.2 Why SQL (PostgreSQL) rather than a document store

The hard parts of this domain are **invariants across rows**: one active evaluation per submission; a result must reference the exact rubric version it was scored with; criterion scores must exist for every criterion before aggregation; a review can be open only once; the job that starts an evaluation must commit together with the evaluation row. Postgres expresses every one of these with constraints and transactions (partial unique indexes, foreign keys, `ON CONFLICT`, `FOR UPDATE SKIP LOCKED`). In MongoDB they become application code that is right until the second instance races the first. The flexible parts (evidence lists, flags, project card, model info) are JSONB, which gives schema flexibility where it is wanted without losing the relational core. Postgres also carries the queue (Phase 1) and the vectors (pgvector), which keeps the first production system to one database engine.

5.3 Important indexes and the invariant they serve

Index	Purpose
`evaluations_one_active_per_submission` (partial unique on `submission_id` WHERE active)	**Duplicate processing is impossible**, independent of application logic.
`UNIQUE (evaluation_id, criterion_id)` on `criterion_scores`	Idempotent per-criterion upsert; resumable evaluate stage.
`UNIQUE (evaluation_id)` on `evaluation_results`	One aggregate per run; retried aggregate upserts.

Index	Purpose
`UNIQUE (evaluation_id, stage, attempt)` on `evaluation_attempts`	Exactly one audit row per delivery.
`human_reviews_one_open_per_evaluation` (partial unique)	At most one open review.
`UNIQUE (assignment_id, version)` on `rubrics`, `UNIQUE (rubric_id, key)` on criteria	Rubric versions are addressable and criteria keys stable for overrides.
`jobs_ready_idx (queue, available_at) WHERE status='ready'`	O(log n) lease query; the table stays small because done jobs are purged.
`jobs_leased_idx (leased_until) WHERE status='leased'`	Cheap reaper scan for crashed workers.
`submissions(student_id, created_at DESC)`, `submissions(assignment_id, status)`	Student dashboard and evaluator filters.
`evaluations(submission_id, created_at DESC)`	History per submission.
`evaluations(status) WHERE status IN ('queued','needs_review')`	Operational dashboards and the review queue.
`notifications(user_id, created_at DESC) WHERE read_at IS NULL`	Unread feed.
`idempotency_keys PRIMARY KEY (scope, key)`	O(1) replay lookup; rows expire after 24–72 h by a cron delete.

5.4 Evaluation history

Re-running an evaluation **inserts a new `evaluations` row**; nothing is overwritten. Each row pins `rubric_id`, stores `model_info` (provider, model ids, prompt version, token usage) and keeps all attempts. This gives: reproducibility ("scored by prompt v2026-10-12.1 on claude-opus-5-5 against rubric v2"), audits for disputes, and a dataset for calibration (§6.5). Rubric edits never alter history because a published rubric is immutable; editing creates a version.

5.5 Preventing inconsistent states

1. **Closed state set** via the `evaluation_status` enum and a transition table; every change is a conditional update that fails (0 rows) when the row is not in an expected state.
2. **Transactions around multi-table changes**: start (evaluation + job + submission status), aggregate (result + status + review + notification), decision (overrides + aggregate + review + status + notification).
3. **Database-enforced uniqueness** for the invariants listed above, so bugs or races degrade to a 409, never to two truths.
4. **Outbox-style enqueue**: the job row is written in the same transaction as the state change. With SQS (Phase 2) the same property is kept by writing an `outbox` row in the transaction and having a relay publish it; the consumer's dedup key and the state machine absorb the rare double publish.
5. **Workers never write status directly**; they go through the same `transition()` helper as the API.

---

6. AI / LLM architecture

6.1 The problem in one sentence

A project can be 10k lines of code, dozens of documents, images, schemas and a video. Sending it to a model in one request is impossible or pointless (context limits, cost, and attention dilution: a 400k-token prompt about a 6-criterion rubric produces vague scores). The design therefore turns the project into a **small, evidence-bearing representation first**, then asks narrow questions.

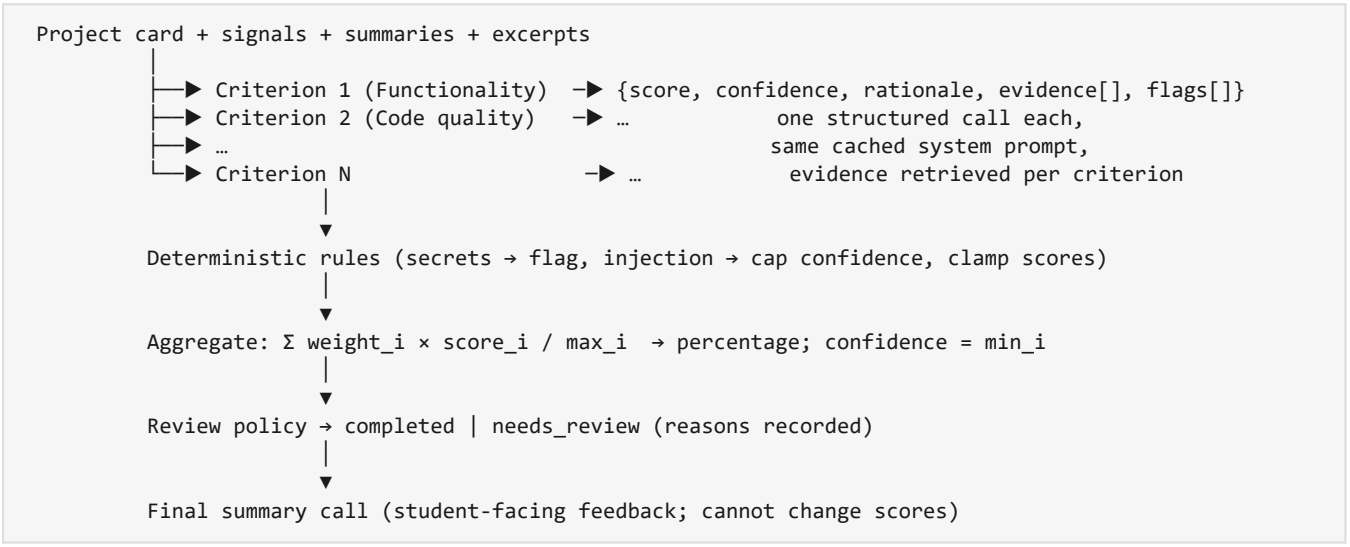
6.2 Project understanding pipeline

```
S3 sources ─▶ EXTRACT ─▶ INVENTORY ─▶ SELECT ─▶ SUMMARISE (map) ─▶ PROJECT CARD (reduce) ─▶ INDEX
              zip/clone    classify     rank by     1 cheap LLM call   1 strong LLM call        embeddings
              bounded,     code/test/   priority,   per file, bounded  over summaries +         per summary
              no exec      docs/schema  skip noise  excerpt (≤24k ch)  README + signals         (pgvector)
                           + signals
```

Step	What happens	Implementation / notes
Extract	ZIP streamed and validated entry by entry; GitHub shallow clone (`--depth 1` , timeout, no prompts) with `.git` removed; PDFs/docs copied under `_documents/` ; video/Drive/live URL recorded as metadata (Phase 2 adapters: transcript via Transcribe, Drive export, headless probe).	`src/pipeline/extract.ts` ; limits in `src/config.ts`
Identify relevant files / ignore irrelevant	Walk without following symlinks; skip `node_modules` , `.git` , `dist` , `venv` , … ; classify by extension/name into code, test, docs, config, schema, data, lock, binary; detect binaries by null bytes; skip files > 512 KB and lock files.	`src/pipeline/inventory.ts`
Deterministic signals	Tests present, README size, CI config, Dockerfile, schema files, entry points, route files, language line counts, **secret patterns**, **prompt-injection phrases**, AI library mentions. Facts computed by tooling are labelled "trustworthy" in prompts; student text is not.	same
Prioritise	README 100 > manifests 90 > entry points 85 > schema 80 > routes/services 75 > tests 60 > docs 55 > other code 50 (penalising tiny and gigantic files) > config. Only the top 40 files get an LLM summary; the rest are listed by name and size.	`priorityOf()`
Chunk	Files are excerpted head+tail to ≤ 24k characters (~6k tokens). Phase 2: syntax-aware chunking with tree-sitter (functions/classes as units) for files that matter and exceed the budget, and page-wise text for PDFs.	`readExcerpt()`
Summarise (map)	One structured call per file with the cheap tier (`claude-haiku-5-5`): purpose, key elements, quality notes, red flags. Concurrency 4 per evaluation; results persisted to S3 as a checkpoint.	`understandStage()`
Project card (reduce)	One strong-tier call over inventory brief + README excerpt + all summaries (bounded to 60k characters): stack, architecture, data model, API surface, testing, documentation, AI usage, strengths, risks. This card is the shared context for every criterion.	`ProjectCardSchema`

Step	What happens	Implementation / notes
Build context / prevent context-window problems	Every prompt is assembled from bounded parts (card ≤ ~3k tokens, signals ≤ 0.5k, ≤10 relevant summaries, ≤4 raw excerpts of 4k chars). Worst case per criterion call ≈ 25k input tokens, far under limits and cheap enough to cache. Token counts are measured with the provider's count endpoint in production, not estimated by characters.	`criterionUser()`
Embeddings / vector search	Phase 1 PoC: lexical ranking of summaries against the criterion text plus per-criterion path hints. Phase 2: embed each file summary (and chunk) with a text-embedding model into `pgvector`, filter by `evaluation_id`, and retrieve top-k per criterion question ("where is error handling?", "how is the schema defined?"). Vectors are scoped to one evaluation and deleted after 30 days, so the index stays small.	`rankEvidence()`

6.3 Evaluation structure



6.4 Choices and reasons

Option	Decision	Why
One LLM call for everything	**No**	Context limits, attention dilution, no per-criterion evidence, one failure loses everything, impossible to retry cheaply.
Multiple specialised calls	**Yes: map (files) → reduce (card) → one call per criterion → summary**	Each call is small, cacheable, retryable and checkpointed. Per-criterion calls produce evidence-cited rationales a student can learn from. Different tiers for different jobs cut cost ~3×.
Agents (model-driven tool use)	**Bounded, later**	An agent that can `grep`, open files and run read-only queries would improve evidence quality for large projects. Risks: unbounded cost/latency and a larger prompt-injection surface (the agent reads hostile text and then chooses actions). Plan: a per-criterion agent with a small tool set (search summaries, read file excerpt), a step budget (≤ 8 tool calls) and a token budget, enabled only when retrieval confidence is low. Not in Phase 1.
RAG	**Yes**	Retrieval of the most relevant summaries/excerpts per criterion is what keeps prompts small and scores grounded. Lexical in the PoC, embeddings in Phase 2 (§6.2).

Option	Decision	Why
Structured outputs	**Yes, everywhere**	Schema-enforced JSON (`messages.parse` + `output_config.format`) removes parsing failures and makes "malformed response" a rare, retryable event. Numeric ranges are still clamped in code: the schema guarantees shape, not semantics.
Deterministic rules + LLM	**Yes**	Tooling computes facts (tests exist, secrets committed, injection phrases, weights sum) and the model judges content. Rules also post-process: cap confidence when injection is suspected, add `secrets_committed`, clamp scores, route extremes to review.
Human-in-the-loop	**Yes, by policy**	Review when: min confidence < 0.6; any criterion flagged `prompt_injection_suspected`, `possible_plagiarism`, `secrets_committed`, `llm_refusal`; > 1 criterion with `insufficient_evidence`; percentage < 35 or > 92; or the student asks. Reviewers can override scores; overrides are recorded with the reviewer id and feed calibration.

6.5 Quality: evaluating the evaluator

- **Golden set**: 50–100 past submissions scored by two humans; measure mean absolute error and agreement (Cohen's κ on grade bands) of the AI per criterion. Any prompt or model change runs the golden set in CI before it ships; results are stored with `prompt_version`.
- **Shadow mode**: a new model/prompt scores live submissions without being shown; drift versus the current version is monitored before switching.
- **Reviewer overrides** are the ongoing labelled dataset: criteria with the largest systematic deltas get their guidance text improved first (the fix is usually in the rubric guidance, not the model).
- **Consistency**: temperature is left at default with adaptive thinking; variance is measured by re-scoring a sample. Where variance matters (borderline grades) the policy routes to a human rather than averaging more samples.

6.6 Cost per evaluation (list prices, Anthropic API, Oct 2026)

Step	Calls	Model	Input tokens	Output tokens	Cost
File summaries	40	Haiku 5.5 ($0.10 / $0.50 per MTok)	240k	16k	$0.032
Project card	1	Opus 5.5 ($4 / $20 per MTok)	20k	1.5k	$0.110
Criterion scoring	6	Opus 5.5	150k	9k	$0.780
Final summary	1	Opus 5.5	6k	1k	$0.044
Total	**48**		**~416k**	**~28k**	**≈ $0.97**

Levers, in the order I would pull them (measured against the golden set, not assumed):

1. **Prompt caching**: the system prompt and the project card + signals prefix are identical across the 6 criterion calls; cache reads cost $0.20/MTok on Opus 5.5, so ~15k × 5 reads saves ≈ $0.30 → **≈ $0.65**.
2. **Content-hash reuse**: students resubmit; a file whose SHA-256 matches a previously summarised file reuses the summary. Typical savings 30–60% of summary tokens on resubmission.
3. **Tier the criterion calls**: Sonnet 5.5 ($2 / $10) for criteria where golden-set agreement is equal (often Documentation, Innovation) → ≈ **$0.40**.
4. **Batch API** (50% discount, hours of latency) for submissions far from a deadline; synchronous for near-deadline ones. The queue already has the priority dimension for this.
5. **Fewer, better summaries**: raise the per-file budget for the top 15 files and drop the long tail to name-only; verify on the golden set.

At 30/day the unoptimised bill is ≈ $30/day. At 10,000/day it is ≈ $9,700/day unoptimised and ≈ $3–4k/day after levers 1–4: still the largest line item by an order of magnitude, which is why §9 treats token economy as the main scaling workstream.

6.7 Prompt-injection defences (summary; details in §8)

Student material is always wrapped in `<project_material>` and described as untrusted data; the system prompt instructs the model to report, not follow, grading instructions found in it; the inventory scans for injection phrases and flags the evaluation; flagged evaluations get confidence capped at 0.5 and go to a human; evidence must cite real paths and the pipeline trusts its own inventory, not the model, for file paths; the final summary call cannot change scores.

---

7. Reliability and failure handling

7.1 LLM failures

Failure	Detection	Handling
Rate limit (429)	SDK `RateLimitError` → `rate_limit`, retryable	Exponential backoff with jitter (1s→30s, 3 tries) inside the call; then stage retry via queue backoff (15s→10min); Phase 2: a shared token-bucket in Redis sized to the provider's TPM/RPM so workers never collectively exceed it; autoscaling of LLM workers is capped by the same budget.
Timeout	SDK timeout (180 s) → `timeout`, retryable	Same retry ladder; long calls stream in production so partial progress is visible and HTTP idle timeouts are avoided.
Unavailable (5xx, 529 overloaded)	`unavailable`, retryable	Retry ladder; after N minutes of elevated 5xx the circuit opens: workers pause LLM stages (jobs stay queued, no data loss), an alarm fires, and optionally the provider flag flips to Bedrock/secondary.
Malformed response	`parsed_output` null → `malformed`, retryable once	Structured outputs make this rare; a retry with the same prompt usually succeeds; persistent failure fails the stage with a clear code.
Token limit exceeded	`stop_reason = max_tokens` or 400 "too long" → non-retryable at call level	Prompts are bounded by construction; if hit, the stage shrinks the context (fewer summaries/excerpts) on the next attempt; the attempt record shows which budget was exceeded.
Refusal (`stop_reason = refusal`)	non-retryable	Criterion gets a zero-confidence placeholder and flag `llm_refusal`; evaluation completes into `needs_review`; a human scores that criterion.
Runaway cost	call counter in `BudgetedLlm`	Hard cap per evaluation (`budget_exceeded`, non-retryable) + daily spend alarm from token metrics.

7.2 Processing failures

Failure	Handling
Corrupt ZIP	`corrupt_archive`, non-retryable → evaluation `failed` with a student-readable message; the student fixes and resubmits.
Invalid GitHub repository	Rejected at `POST /submissions` by URL pattern; at clone time `clone_failed` is retryable (GitHub hiccups) up to max attempts, then failed with the git error summary (private repo, deleted, …).

Failure	Handling
Large repository	GitHub API size check before clone (Phase 2); shallow clone; extracted size limit (2 GiB) → `too_large`; worker disks are sized for it and the work dir is deleted after aggregation.
Unsupported file	Inventoried, never read; the project card states "not evident" rather than guessing; unsupported *source kinds* are rejected at creation.
Worker crash	Lease expiry → redelivery → idempotent re-run (§3.5). Graceful shutdown on SIGTERM stops leasing and finishes in-flight jobs (30 s grace, lease still protects beyond that).
Network failure (S3, DB, GitHub)	Retryable with backoff; DB connection errors are retried by the pool; stage retries cover the rest. Postgres failover (Multi-AZ) is ~60 s, inside the retry window.
Poison message	After `max_attempts` the job is dead-lettered (`status='dead'`, SQS DLQ), the evaluation is `failed` with the last error, an alarm fires, and `GET /admin/queue` lists it for replay after a fix.

7.3 Duplicate processing and idempotency

`POST /evaluate` called five times (same client retrying, or five tabs) resolves as follows:

Case	Outcome
Same `Idempotency-Key`, same body	Call 1 runs; calls 2–5 return the **stored 202 response** with `Idempotent-Replayed: true`. If one arrives while call 1 is still inside the transaction: 409 `idempotency_in_progress`, retry safe.
Same key, different body	422 `idempotency_key_reused`.
No key	Call 1 creates the evaluation; calls 2–5 hit the partial unique index and get **409** **`evaluation_in_progress`** with the active evaluation id.
Key scope	`(scope = "evaluations:<userId>", key)`: users cannot collide with or replay each other's requests. Rows are purged after 72 h.
Worker side	Dedup key per stage message; state-machine guard on delivery; `ON CONFLICT` writes. A fifth copy of a stage message is acknowledged and dropped.

Implementation: `src/api/idempotency.ts` (insert-first with `ON CONFLICT DO NOTHING`, then run, then store response; on handler error the key row is deleted so the client can retry) and `db/migrations/001_init.sql` (`evaluations_one_active_per_submission`).

7.4 Observability that makes the above operable

- Every log line carries `evaluation_id`, `submission_id`, `stage`, `attempt`, `worker_id`, `request_id`.
- Metrics: queue depth and **age of the oldest ready job** (the primary SLO signal), stage duration p50/p95, failures by `error_code`, LLM calls/tokens/cost per evaluation, 429 rate, DLQ count, review queue size.
- Alarms: oldest job age > 15 min; DLQ > 0; failure rate > 5% over 15 min; LLM 5xx/429 spikes; daily LLM spend > budget.
- Traces: one trace per evaluation spanning API → worker stages → LLM calls (OpenTelemetry), so a slow evaluation shows which call was slow.

---

8. Security

Students upload untrusted code and documents; the system processes them with privileged credentials (LLM keys, DB access). The design assumes every upload is hostile.

Risk	Mitigation
Authentication	Password login with scrypt (argon2id in production), JWT with 12 h expiry and short refresh in Phase 2 via an IdP; staff accounts behind SSO + MFA; tokens never logged (pino redaction).
RBAC	Three roles enforced in `preHandler` hooks; ownership checked in SQL (`student_id = $user`) for every student read; staff-only routes (`/decision` , `/reviews` , `/admin/*`). Results for `needs_review` evaluations hidden from students until decided.
File upload security	Direct-to-S3 presigned PUT with size cap (1 GiB) and 15-minute expiry; server-side verification of size and checksum; **magic-byte sniffing** against the declared kind; filenames sanitised; objects stored under `submissions/<id>/<sourceId>/` with no user-controlled path components; bucket private, encrypted, no public ACLs.
Malware	Nothing from a submission is executed in Phase 1 (no `npm install` , no build). Extraction and static analysis run in a worker container with a non-root user, read-only root filesystem, no inbound network, egress only to S3/DB/LLM/GitHub via an allow-list. Optional ClamAV scan on upload completion. Phase 2 code execution only inside a microVM (§8.4).
ZIP bombs	Streaming extraction; caps on entry count (20k), per-entry size (50 MiB), total extracted size (2 GiB) and **compression ratio (100:1 for entries > 1 MiB)**; actual bytes counted against declared size while inflating, so a lying header aborts (`src/pipeline/extract.ts` , tested in `tests/extract.test.ts`).
Path traversal	Entry names validated (no absolute paths, no `..` , no backslashes); the resolved path must stay under the target directory; **symlink entries are skipped** (a symlink then a write through it is the classic bypass); object keys are validated against a strict charset and resolved under the store root.
SSRF	Student URLs are validated at creation (scheme, no credentials, no loopback/private/link-local/metadata addresses, no `.internal/.local`); at fetch time DNS is resolved and every address re-checked, redirects re-validated per hop, and all outbound fetches go through an egress proxy with a deny-list for RFC1918 and 169.254.169.254. The live-URL probe runs in the sandboxed worker, never in the API.
GitHub repository access	Public repos: anonymous shallow HTTPS clone, `GIT_TERMINAL_PROMPT=0` , timeout, `.git` removed, size checked. Private repos (Phase 2): a GitHub App installed by the student on their repo; short-lived installation tokens scoped to that repo, never stored long-term; no SSH keys, no personal tokens from students.
Secrets / API keys	AWS Secrets Manager / SSM Parameter Store injected at task start; IAM task roles instead of static AWS keys; LLM key rotated and never in images or logs; `.env` only for local dev and git-ignored; secret scanning in CI (gitleaks) on *our* repo; **student secrets detected** and flagged (`secrets_committed`) rather than used.
LLM prompt injection	Student text is data: wrapped in `<project_material>` , described as untrusted in the system prompt; grading instructions found inside it are reported as `prompt_injection_suspected` ; the inventory independently scans for injection phrases; flagged evaluations get confidence ≤ 0.5 and a human review; the model cannot call tools in Phase 1 (no action surface); evidence paths are taken from the inventory, not trusted from the model; the final summary step cannot alter scores. Demonstrated by `npm run demo -- --inject` .
Student data isolation	Every query is tenant-filtered by `student_id` /role; S3 prefixes per submission; presigned GET URLs for downloads expire in minutes; artifacts (summaries, cards) are per evaluation and deleted on a retention schedule; LLM provider configured with no training on our data and zero/short data retention where available.

Risk	Mitigation
Access to evaluation results	Students see only their own, only when `completed` / `approved`; evaluators see all; audit log records every read of a result by staff (`audit_log`), every decision and override.
Rate limiting	ALB/WAF rate rules per IP; API per-user limits on expensive endpoints (create submission, start evaluation, upload completion) with `@fastify/rate-limit` (Redis store in Phase 2); per-user daily quota of evaluations; global backpressure (§9.7) protects the LLM budget.
Transport / headers	TLS at the ALB, HSTS, `x-content-type-options: nosniff`, `cache-control: no-store` on API responses, strict CORS to the web origin.

8.4 Running student code (Phase 2)

Functionality carries 25% of the grade, and static evidence is weaker than running the tests. When this is added it runs **only** in a Firecracker microVM (or gVisor-sandboxed container) per evaluation: no network (dependencies from an internal mirror), CPU/memory/time limits, read-only base image, results (test output, coverage) copied out as text and fed to the Functionality criterion as evidence. The orchestrator treats the sandbox as untrusted output too (it is still student code talking).

---

9. Scaling: 30 → 10,000 submissions/day

The principle: change what a measured signal says is the bottleneck, in this order, and keep the interfaces (`JobQueue`, `ObjectStore`, `LlmProvider`) so each change is local.

Area	Phase 1 (30/day)	Trigger to change	Phase 2 (~1,000/day)	Phase 3 (10,000/day)
API	2 ECS tasks behind ALB	p95 > 300 ms or CPU > 60%	Autoscale 2–6 on CPU/req count; presigned uploads already keep the API thin	6–20 tasks; read replicas for dashboards; CDN for the SPA; SSE endpoint moves to a separate "realtime" service fed by Redis pub/sub
Workers	1–2 tasks, concurrency 2–4, one queue	Oldest ready job age > 5 min sustained	Split queues: `extract` (disk/CPU-heavy, few concurrent) vs `llm-stages` (I/O-bound, concurrency 8–16 per task); autoscale on queue age	30–70 worker tasks at peak on ECS capacity providers (Spot for LLM stages, On-Demand for extract); per-course priority lanes; scale-to-zero overnight
Queue	Postgres table, `SKIP LOCKED`	> ~50k jobs/day, or desire to isolate queue from DB load	**SQS standard + DLQ** via the same `JobQueue`; transactional outbox relay for enqueue	SQS with multiple queues (priority: deadline-near vs batch); SQS FIFO not needed (ordering is per evaluation and enforced by the state machine)

Area	Phase 1 (30/day)	Trigger to change	Phase 2 (~1,000/day)	Phase 3 (10,000/day)
Database	RDS Postgres Multi-AZ, db.t4g.medium	CPU > 60%, connections > 70%, slow queries	Bigger instance; PgBouncer; purge `jobs` done rows daily; `idempotency_keys` TTL	r6g class; read replica for evaluator dashboards and analytics; partition `evaluation_attempts`, `audit_log`, `notifications` by month; move raw `evidence` JSON to S3 if row size bites. Writes are ~10k evaluations × ~20 rows/day = trivial for Postgres; the risk is unbounded history tables, hence partitioning and retention.
Object storage	S3 standard	Cost report	Lifecycle: raw uploads → Infrequent Access at 30 days → Glacier at 90; artifacts deleted at 30 days (results in DB stay)	Same; 1 TB/day ingest; S3 Transfer Acceleration for large uploads; multipart mandatory > 100 MB
LLM throughput	One provider key, per-worker concurrency limit	429 rate > 1%, oldest job age rising while workers idle	Redis token-bucket shared across workers sized to provider TPM/RPM; request higher limits; prompt caching on; content-hash summary reuse	Multi-provider (Anthropic API + Bedrock) with weighted routing; **batch API for non-deadline submissions** (50% cheaper, async); model tiering per criterion from golden-set data; summaries only for files changed since last submission
Rate limits	ALB + per-user API limits	Abuse or budget alarms	Per-user daily evaluation quota; per-course quota	Admission control: when LLM budget for the hour is exhausted, new evaluations are accepted (202) but scheduled into the batch lane with an honest ETA
Cost	≈ $30/day LLM, ≈ $300/month infra	Cost per evaluation dashboard	Caching + reuse → ≈ $0.65/eval	Tiering + batch → ≈ $0.35–0.45/eval ⇒ ≈ $3.5–4.5k/day LLM; infra ≈ $6–10k/month (ECS, RDS r6g + replica, S3, NAT). Cost per evaluation is the KPI reported to the business weekly.
Caching	Prompt caching	Resubmission rate	Summary cache by file SHA-256 (table `file_summaries(sha256, model, prompt_version) → summary`); project card reuse when the file set is unchanged	Embedding cache by SHA-256; CDN for static; HTTP caching of immutable results (`ETag`)
Monitoring	CloudWatch logs/metrics/alarms	Team size	Grafana dashboards; OpenTelemetry traces; on-call rota	Per-course and per-model quality dashboards (golden-set drift), cost anomaly detection, SLO burn-rate alerts
Backpressure	Queue grows; API still accepts	Oldest job age SLO breached	Autoscale workers on queue age with a ceiling set by LLM budget; shed load by routing to batch lane	Priority lanes (deadline < 24 h first), per-course fair share, explicit "estimated completion" surfaced to students, and a kill switch that pauses LLM stages without losing jobs

9.1 What does *not* change

The data model, the API contract, the state machine, the stage boundaries, the prompts, and the three interfaces. The team should be able to move from Phase 1 to Phase 3 without a rewrite; every row above is a configuration or an adapter.

9.2 Why not Kubernetes everywhere

Two services and a queue do not need a cluster to operate. ECS gives scheduling, health checks, rolling deploys and autoscaling with a fraction of the operational surface. Kubernetes becomes reasonable when the organisation already runs it for other workloads or when the sandbox fleet (§8.4) needs node-level control (Firecracker on bare-metal EC2); even then the API/worker services could stay on ECS.

---

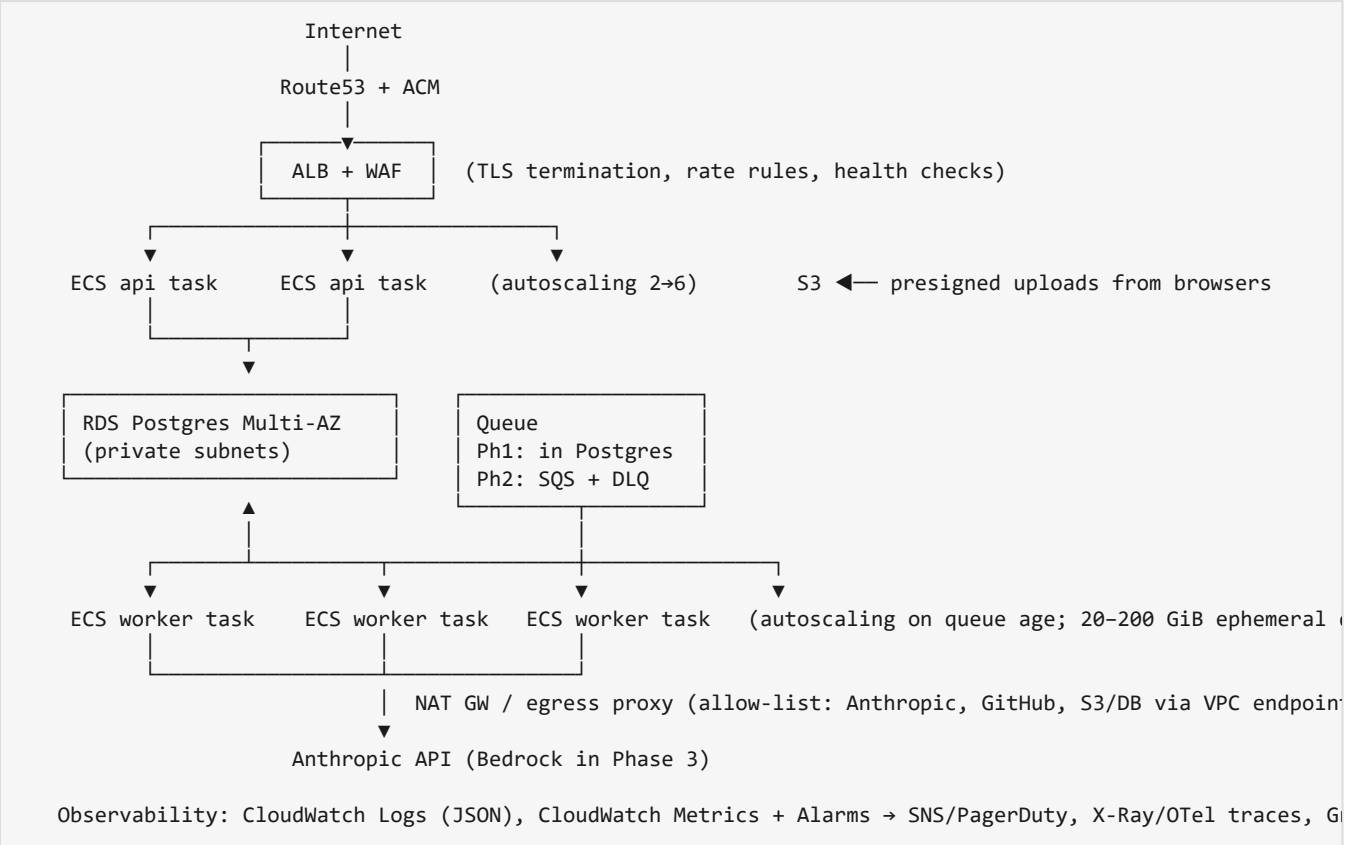
10. Deployment architecture

10.1 Stage A: first deployment (weeks 1–2) — one EC2 host

```
Internet ─▶ Route53 ─▶ EC2 (t3.large, Docker Compose)
                       ├─ nginx  (TLS via certbot, reverse proxy, 1 GiB body limit → /uploads only, rate l
                       ├─ api    (container, port 4200, not exposed publicly)
                       └─ worker (container, concurrency 2)
           RDS Postgres (Multi-AZ)      S3 bucket       Secrets Manager / SSM
```

This is the quickest honest production: one host, managed database, managed storage. It is what `docker-compose.yml` in the repo deploys, plus NGINX and RDS instead of the local Postgres. Known limits: single point of failure for compute, manual scaling. Fine for 30/day while the pipeline is tuned.

10.2 Stage B: Phase 1 production — ECS Fargate



10.3 Operational elements

Element	Approach
Docker	One multi-stage image (`Dockerfile`): build with dev deps, prune, run as the unprivileged `node` user on `node:22-bookworm-slim` with `git` installed; API and worker differ only in `CMD`. Workers add a read-only root FS and a writable `/data/work` volume. Images pinned by digest; Trivy scan in CI.
CI/CD	GitHub Actions: `typecheck` → `test` (Postgres service container) → build image → push to ECR tagged with the git SHA → run migrations as a one-off ECS task (`node dist/db/migrate.js`) → deploy `api` (rolling, min healthy 100%, deployment circuit breaker with automatic rollback) → deploy `worker` (rolling; tasks drain by finishing leased jobs). Staging is deployed on every merge to `main`, production on a tag with a manual approval gate.
Environment management	Three AWS accounts (dev, staging, prod) under one organisation; identical Terraform modules with per-env variables; no shared databases; feature flags (e.g. `LLM_PROVIDER`, batch lane) via SSM parameters read at boot.
Secrets	Secrets Manager for DB password, JWT secret, LLM keys; injected into task definitions as secrets, never as plain env; rotation for DB via RDS integration; IAM task roles for S3/SQS/Bedrock so no AWS keys exist.
AWS services	ALB, WAF, ECS Fargate (+ EC2 capacity providers in Phase 3), ECR, RDS Postgres, S3, SQS (Phase 2), ElastiCache Redis (Phase 2), Secrets Manager/SSM, CloudWatch, SNS, Route53, ACM, VPC endpoints (S3, SQS, Secrets Manager, Bedrock), Transcribe (video).
Logging	pino JSON to stdout → CloudWatch Logs with 30-day retention (90 for audit); structured fields enable Logs Insights queries by `evaluation_id`; PII and secrets redacted at source.
Monitoring	CloudWatch metrics from the app (EMF) + container insights; dashboards: throughput, queue age, stage latency, failure codes, LLM cost; alarms in §7.4; synthetic canary that runs a tiny evaluation hourly against staging.

Element	Approach
Health checks	`GET /healthz` (process alive) for the ALB; `GET /readyz` (DB reachable, queue stats) for deploy gating; worker liveness = recent heartbeat row per worker id (an ECS health command checks `last_heartbeat < 60 s`).
Rollbacks	Images are immutable and tagged by SHA; rollback = redeploy the previous task definition (one command, or automatic via the circuit breaker). Database migrations follow **expand/contract**: additive first (new columns nullable, new tables), deploy code that works with both shapes, remove old shapes a release later. The queue payloads are versioned so old workers ignore unknown stages.

---

11. Technology decisions and alternatives

Full table as requested, plus two decisions that mattered in practice.

Decision	Options considered	Choice	Reason	Sources
Database	PostgreSQL vs MongoDB	**PostgreSQL**	The domain is relational with cross-row invariants (one active evaluation, pinned rubric versions, complete criterion sets). Partial unique indexes, transactions and `SKIP LOCKED` do work that would otherwise be fragile application code. JSONB covers the flexible parts. One engine also hosts the Phase 1 queue and pgvector. MongoDB's strengths (schema-less documents, horizontal sharding) are not the constraint at 10k/day × ~20 rows.	PostgreSQL docs: partial indexes, SELECT … FOR UPDATE SKIP LOCKED, MongoDB transactions
Queue	SQS vs RabbitMQ vs Kafka (vs Postgres table)	**Postgres table now → SQS at scale**	Phase 1 volume is tiny; a DB queue gives transactional enqueue and zero extra services. SQS is the step up: managed, visibility timeout and DLQ map 1:1 to the lease/retry model, scales without ops. RabbitMQ needs a cluster to run and offers nothing we need beyond SQS. Kafka is a log for streams and replay; we need a work queue with per-message leases and dead-lettering, which Kafka does not provide natively.	SQS visibility timeout, SQS dead-letter queues, RabbitMQ docs, Kafka design, Transactional outbox

Decision	Options considered	Choice	Reason	Sources
LLM	Managed API (Anthropic, OpenAI, Bedrock-hosted) vs self-hosted open-source	**Managed API: Claude via the Anthropic SDK, Bedrock route at scale; provider interface keeps it swappable**	Code evaluation needs strong reasoning over long, messy inputs; frontier managed models win on quality today and remove GPU operations entirely. Needed features exist: structured outputs, prompt caching, 1M context, batch API. Bedrock offers the same models with IAM auth, VPC endpoints and consolidated billing when the AWS footprint matters. Self-hosting (Llama/Qwen class) would only pay off for the summariser tier at very high volume, and Haiku-class pricing makes even that marginal; revisit with real volume data.	Anthropic structured outputs, Prompt caching, Message Batches, Anthropic pricing, Amazon Bedrock, OpenAI structured outputs
Vector DB	pgvector vs Pinecone vs Weaviate	**pgvector** (Phase 2), dedicated store only for cross-submission search	Retrieval here is *scoped to one evaluation* (a few hundred vectors filtered by `evaluation_id`), which is trivial for pgvector with an HNSW index and keeps vectors transactional with the rows they describe. Pinecone/Weaviate add a service, a network hop and a sync problem for no gain at this access pattern. If plagiarism detection across all submissions (global ANN over tens of millions of vectors) becomes a requirement, that is a different workload where a dedicated engine (or OpenSearch k-NN) is justified.	pgvector, Pinecone docs, Weaviate docs
Compute	EC2 vs containers (ECS/EKS) vs serverless (Lambda)	**Containers on ECS** (Fargate first, EC2 capacity providers for workers at scale); EC2 + Compose only for the first weeks	Evaluations run 2–15 minutes with large local disks and long-lived LLM connections: Lambda's 15-minute cap, 10 GB ephemeral storage and per-ms pricing fit badly. Plain EC2 means patching and hand-rolled deploys. ECS gives rolling deploys, health checks and autoscaling without running a control plane; Fargate removes hosts entirely; EC2 capacity providers return when Spot savings or Firecracker sandboxes need node control. EKS only if the org already runs Kubernetes.	ECS developer guide, Fargate ephemeral storage, Lambda quotas, Firecracker
Language/runtime	Node.js/TypeScript vs Python	**TypeScript** for API and workers	One language, shared types for job payloads and schemas, Zod for both HTTP validation and LLM output schemas, excellent async I/O for an LLM-bound pipeline. Python's parsing/ML ecosystem is not needed because the model, not local libraries, does the understanding; tree-sitter and PDF text extraction exist for Node when needed.	Fastify, Zod, tree-sitter

Decision	Options considered	Choice	Reason	Sources
Orchestration of the pipeline	Hand-rolled stage jobs vs Celery/BullMQ vs Temporal/Step Functions	**Stage jobs over a plain queue, state in Postgres**	The workflow is a fixed four-step line with per-step checkpoints; a plain queue plus a state machine is simpler to reason about and to operate than a workflow engine, and the PoC proves the recovery semantics. Temporal or Step Functions become attractive if the graph grows branches (sandbox execution, transcription, re-scoring) and humans need to see workflow histories; the stage boundary design makes that migration straightforward.	Temporal docs, AWS Step Functions, BullMQ
Safe extraction	Trust the archive vs validate	**Validate every entry while streaming**	Zip bombs, path traversal and symlink tricks are standard attacks against upload pipelines.	OWASP File Upload Cheat Sheet, OWASP Top 10 for LLM Applications

---

12. Implementation roadmap

Phase	Weeks	Deliverables	Exit criteria
0. Foundation (done in PoC)	0–1	Schema, API skeleton with auth/RBAC, idempotent start, Postgres queue, worker loop with leases, extraction with limits, inventory + signals, LLM provider interface + mock, staged pipeline, review routing, tests, Docker.	`npm run demo` green; 31 tests green.
1. Production MVP (30/day)	2–5	S3 presigned uploads + completion callback; Anthropic provider live with structured outputs and caching; prompts tuned on a 30-item golden set; evaluator console endpoints; CI/CD to ECS staging/prod; CloudWatch dashboards and alarms; Stage A/B deployment; retention jobs (idempotency keys, jobs, artifacts).	p95 time-to-result < 20 min; MAE vs human ≤ 1.0 point per criterion on the golden set; zero duplicate evaluations in load test.
2. Quality & sources	6–10	PDF/doc text extraction; video transcription adapter; Drive adapter; live-URL safe probe; pgvector retrieval; summary cache by file hash; GitHub App for private repos; SSE status via Redis; rate limits with Redis; shadow-mode evaluation harness.	All source kinds produce evidence; retrieval improves golden-set agreement; review rate < 15%.
3. Scale (1k → 10k/day)	11–16	SQS + outbox; split extract/LLM worker pools with queue-age autoscaling; shared LLM rate limiter; batch lane and priority lanes; model tiering per criterion from data; Bedrock route; partitioning and read replica; cost dashboards; sandboxed test execution (Firecracker) behind a flag.	Sustain 35 evaluations/min at peak with oldest-job age < 10 min; cost per evaluation ≤ $0.45; no SLO burn during a provider incident (graceful degradation to batch).

---

13. Use of AI tools (disclosure)

As permitted by the brief, I used AI tools during this assignment:

- **Claude (Anthropic), through Claude Code in VS Code**, was used to draft and iterate this document and the proof-of-concept code from my requirements and design direction, to look up current API shapes (Anthropic SDK structured outputs and pricing), and to generate the test cases for the extraction and scoring logic. I reviewed the design decisions, the schema and the code, ran the tests and the end-to-end demo locally (`npm test`, `npm run demo`, `npm run demo -- --inject`), and corrected failures found during that run (state-machine terminal set, zip entry-name error mapping, test isolation).
- No AI tool had access to AlmaBetter systems or data; all examples are synthetic.
- All architectural choices in §11 are mine and I am prepared to defend or revise them in discussion.

---

14. References

Official documentation and articles used while researching this design:

- PostgreSQL: Partial indexes · FOR UPDATE SKIP LOCKED · JSONB · Table partitioning · pgvector
- Queues: Amazon SQS developer guide · Visibility timeout · Dead-letter queues · RabbitMQ · Apache Kafka design · Transactional outbox pattern
- Idempotency: Stripe: Idempotent requests · IETF draft: The Idempotency-Key HTTP Header Field
- LLM: Anthropic structured outputs · Prompt caching · Message Batches API · Anthropic pricing · Amazon Bedrock · OpenAI structured outputs · OWASP Top 10 for LLM Applications
- Vector search: Pinecone docs · Weaviate docs · OpenSearch k-NN
- Compute & deployment: Amazon ECS developer guide · Fargate task storage · ECS deployment circuit breaker · AWS Lambda quotas · S3 presigned URLs · S3 lifecycle · AWS Secrets Manager · Firecracker · gVisor
- Security: OWASP File Upload Cheat Sheet · OWASP SSRF Prevention Cheat Sheet · GitHub Apps · ClamAV
- Tooling: Fastify · Zod · tree-sitter · OpenTelemetry · Temporal · AWS Step Functions